C. Context Propagation & Request Lifecycle

Standalone capability brief · Suggested module: **Request Context SDK**

Purpose

Carry trusted request metadata consistently across API calls, service-to-service calls, queues, scheduled jobs, and background workers while preserving deadlines, identity, and traceability.

1. Scope and capability areas

Correlation and tracing

Propagate request IDs, correlation IDs, and trace context across synchronous and asynchronous boundaries.

Tenant and actor context

Represent tenant, organization, authenticated subject, initiating actor, and delegated actor separately.

Locale and region

Carry locale, time zone, and regional routing context where relevant.

Deadlines and cancellation

Propagate request deadlines and cancellation signals so downstream work does not continue needlessly after the caller has abandoned it.

Trusted identity

Distinguish authenticated workload identity from user-supplied metadata. Define how delegated authorization context is validated.

Async propagation

Serialize an allowlisted context envelope into messages and scheduled work, with validation at consumption time.

2. Shared contracts and standards

- Use a versioned context schema with required, optional, and prohibited fields.
- Never trust tenant IDs or actor claims solely because they arrived in a header or message payload.
- Define context propagation rules for HTTP, RPC, events, jobs, and scheduled tasks.
- Keep secrets, access tokens, and unnecessary personal data out of context envelopes.

3. Recommended implementation approach

- Provide language-specific middleware and SDK adapters for incoming requests, outgoing calls, and message handlers.
- Generate trace and request identifiers at trusted boundaries when absent.
- Use allowlists for fields crossing trust boundaries and validate all propagated values.
- Set explicit deadline defaults and maximums; distinguish an absent deadline from an unlimited deadline.
- Define how context is reconstructed for scheduled work that has no live initiating request.

4. Failure behavior and operational expectations

- If optional context is malformed, reject or discard it according to the field's trust level and emit diagnostics.
- If a deadline expires, cancel downstream work and return a standardized timeout outcome.
- If tenant or actor context required for authorization is missing, fail closed rather than guessing.

5. Suggested operational metrics

- Percentage of requests with complete trace linkage
- Missing or invalid tenant context incidents
- Work canceled after deadline
- Context schema compatibility errors

6. Readiness and acceptance checklist

- ■ A trace can be followed across API, queue, and worker boundaries.
- ■ Tenant and actor values are validated at trust boundaries.
- ■ Deadlines and cancellation work in synchronous and asynchronous execution.
- ■ Context propagation does not leak credentials or unnecessary personal data.

Ownership note: assign a named team, service owner, support route, versioning policy, and deprecation process before treating this capability as a platform contract.